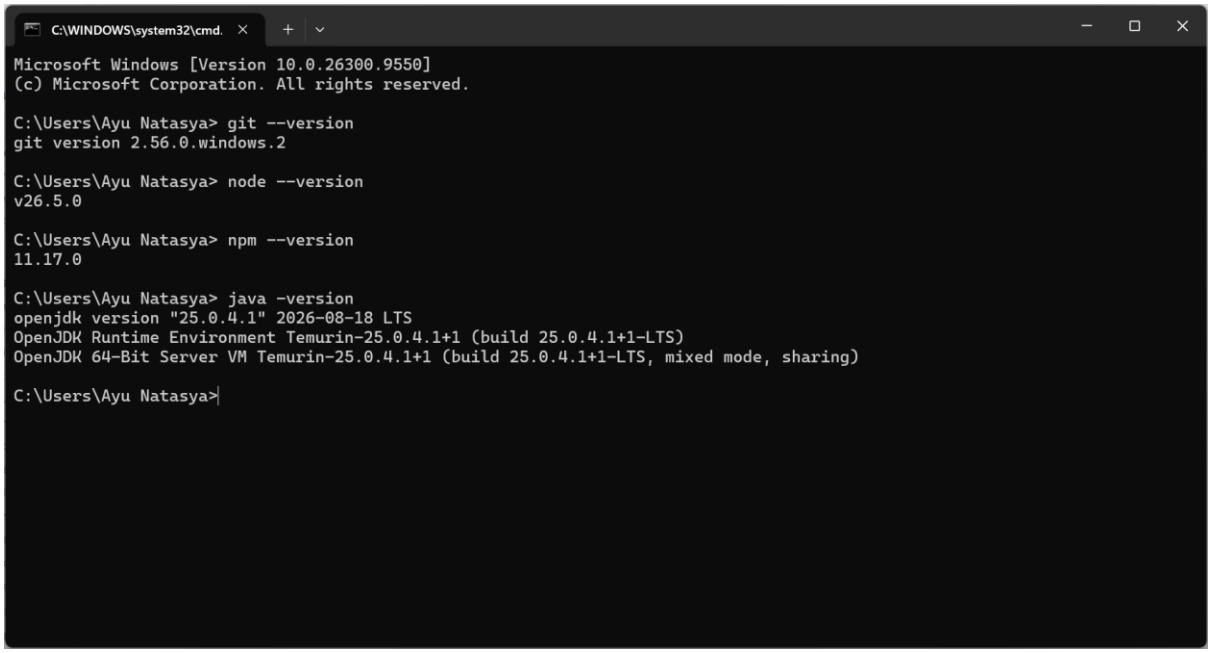
Nama: Ayu Natasya

NIM: 240511018

Kelas: TI24A

Mata Kuliah: Pemrograman Berbasis Platform



Aspek	Web	Desktop	Mobile
Bahasa Utama	- Sisi client (front-end) berjalan di browser dan memakai tiga bahasa dasar, yaitu HTML untuk struktur halaman, CSS untuk tampilan, dan JavaScript untuk interaksi. Di atasnya sering dipakai library atau framework seperti React, Vue, atau Bootstrap supaya pengerjaan lebih cepat. - Sisi server (back-end) lebih bebas memilih bahasa. Beberapa yang umum adalah PHP, Python, JavaScript. Bahasa-bahasa ini bertugas memproses request, menjalankan	C# dengan WPF atau Windows Forms cukup populer, dan C++ dengan Win32 atau Qt dipakai untuk aplikasi yang butuh performa tinggi. Java bisa dipakai lewat JavaFX atau Swing, sedangkan Python bisa dipakai lewat Tkinter atau PyQt untuk aplikasi yang lebih sederhana. Ada juga pendekatan lain, yaitu membuat aplikasi desktop dengan teknologi web memakai Electron, sehingga developer web bisa membuat aplikasi desktop memakai HTML, CSS, dan JavaScript.	Untuk aplikasi native Android, bahasa utamanya Kotlin (yang sekarang paling disarankan) dan Java. Untuk iOS, bahasanya Swift. Karena membuat dua versi terpisah itu memakan waktu, banyak developer memilih framework lintas platform seperti Flutter yang memakai Dart, atau React Native yang memakai JavaScript, sehingga satu kode bisa dijalankan di Android dan iOS.

	logika aplikasi, dan berhubungan dengan database seperti MySQL atau PostgreSQL.		
Arsitektur	Aplikasi web memakai model client-server. Browser sebagai client mengirim request lewat protokol HTTP atau HTTPS, lalu server memproses request itu dan mengirim balik response, biasanya berupa halaman HTML atau data dalam format JSON. Pada aplikasi yang lebih besar, struktur di server biasanya dipisah menjadi beberapa lapisan, yaitu tampilan, logika bisnis, dan data. Pola MVC (Model, View, Controller) yang dipakai Laravel adalah salah satu contohnya. Komunikasi antar bagian sering memakai REST API, sehingga bagian front-end dan back-end bisa dikembangkan terpisah.	Aplikasi desktop berjalan langsung di sistem operasi, jadi memakai CPU, memori, dan penyimpanan komputer pengguna tanpa perantara browser. Bentuknya biasanya satu aplikasi utuh yang berisi tampilan sekaligus logika programnya. Kalau aplikasinya perlu berbagi data antar pengguna, misalnya aplikasi kasir di toko dengan beberapa komputer, aplikasi desktop akan terhubung ke database di jaringan atau ke server lewat API. Jadi arsitekturnya bisa berupa aplikasi mandiri (standalone) atau client-server, tergantung kebutuhan.	Aplikasi mobile berjalan di atas sistem operasi ponsel dan mengikuti aturan siklus hidup (lifecycle) yang ditentukan OS. Contohnya, saat pengguna pindah ke aplikasi lain atau layar terkunci, aplikasi bisa dijeda atau dihentikan oleh sistem untuk menghemat baterai dan memori, sehingga developer harus menyimpan kondisi aplikasi dengan benar. Untuk data yang berasal dari internet, aplikasi mobile biasanya memanggil back-end lewat REST API. Artinya, satu layanan bisa punya satu back-end yang sama dan dipakai oleh versi web, versi mobile, bahkan versi desktop sekaligus.
Keunggulan	- -Bisa dibuka dari perangkat apa saja yang punya browser dan koneksi internet, baik laptop, komputer kampus, maupun ponsel. - Pengguna tidak perlu memasang aplikasi, jadi hambatan untuk mulai memakai sangat kecil. - Pembaruan cukup dilakukan di satu tempat, jadi semua pengguna selalu memakai versi yang sama.	- Performanya umumnya paling baik karena langsung memakai sumber daya komputer. - Aplikasi desktop juga punya akses yang luas ke file, printer, USB, dan perangkat keras lain, dan banyak di antaranya tetap bisa dipakai tanpa internet. Karena itu cocok untuk pekerjaan berat seperti editing video, desain grafis, pemrograman, dan	- Aplikasi mobile bisa memakai fitur ponsel seperti kamera, GPS, sensor, sidik jari, dan notifikasi push. - Ponsel selalu dibawa pengguna sehingga aplikasinya bisa dipakai kapan saja dan di mana saja, dan jumlah pengguna ponsel sangat besar sehingga

	- Mudah dibagikan hanya dengan mengirim link. - Cocok untuk aplikasi yang datanya dipakai banyak orang secara bersamaan, seperti sistem informasi, toko online, dan portal.	pengolahan data besar. - Antarmukanya juga bisa dibuat kaya fitur, misalnya banyak jendela, pintasan keyboard, dan menu yang kompleks.	jangkauan pasarnya luas. Dengan Flutter atau React Native, satu kode bisa dipakai untuk Android dan iOS sehingga lebih hemat waktu dan biaya.
Keterbatasan	- Aplikasi web hampir selalu butuh koneksi internet - Kalau koneksi lambat pengalaman pengguna ikut terganggu. - Akses ke perangkat keras dibatasi oleh browser demi keamanan, dan tampilan serta perilakunya bisa sedikit berbeda antar browser sehingga perlu diuji di beberapa browser. - Performanya umumnya di bawah aplikasi native, terutama untuk proses yang berat. Selain itu, server harus dijaga keamanannya dan harus kuat menghadapi banyak pengguna sekaligus.	- Aplikasi harus dipasang di setiap komputer dan diperbarui satu per satu. - Aplikasi native biasanya hanya untuk satu sistem operasi, jadi kalau mau jalan di Windows, macOS, dan Linux perlu versi yang berbeda atau framework lintas platform. - Aplikasi juga tidak bisa dibuka dari perangkat yang belum terpasang aplikasinya, dan aplikasi berbasis Electron cenderung memakai memori yang cukup besar. - Pengguna juga harus mengunduh dari sumber yang tepercaya, karena risiko file berbahaya lebih besar dibanding web.	- Aplikasi harus lolos review toko dan mengikuti aturan yang berlaku, sehingga rilis versi baru kadang tidak instan. - Ukuran layar dan versi sistem operasi sangat beragam, terutama di Android, sehingga perlu banyak pengujian. - Baterai, memori, dan penyimpanan ponsel juga terbatas. - Pengembangan aplikasi iOS memerlukan perangkat Apple (macOS) dengan Xcode, dan kalau membuat versi native, Android dan iOS ditulis terpisah sehingga biaya dan waktunya lebih besar.
Contoh Aplikasi	- Tokopedia (situs web) untuk belanja online. - Wikipedia sebagai ensiklopedia daring. - Google Docs untuk mengetik dokumen bersama-sama lewat browser.	- Microsoft Word untuk mengetik dokumen. - Visual Studio Code sebagai editor kode (dibuat dengan teknologi Electron).	- WhatsApp untuk berkirim pesan. - Gojek untuk transportasi dan layanan antar. - Instagram untuk media sosial.

	- Canva versi web untuk desain grafis. - - Portal akademik kampus untuk melihat jadwal dan nilai.	- - Adobe Photoshop untuk mengolah gambar.	- Traveloka untuk pesan tiket dan hotel. - Aplikasi mobile banking untuk transaksi perbankan.

REFERENSI

Aalto University. (t.t.). Architecture: Client-server model. FITech101. https://fitech101.aalto.fi/web-software-development/2-internet-and-http/4-architecture-client-server-model

Google. (t.t.). App fundamentals. Android Developers. https://developer.android.com/guide

Microsoft. (t.t.). Build desktop apps for Windows. Microsoft Learn. https://learn.microsoft.com/windows/apps/desktop

Mozilla Developer Network. (t.t.). Learn web development. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn

REFLEKSI

Menurut saya platform yang paling menarik untuk dipelajari adalah web. Saya sudah beberapa kali membuat aplikasi web dengan Laravel, jadi rasanya lebih nyambung dan tidak mulai dari nol. Web juga enak karena hasilnya langsung bisa dilihat di browser, dan kalau ada perubahan cukup refresh halaman. Selain itu, ilmu web bisa dipakai di platform lain, misalnya Electron untuk desktop atau React Native untuk mobile.

Tantangan terbesar saat setup environment menurut saya ada di instalasi dan konfigurasi tools. Git, Node.js, dan JDK harus terbaca di terminal, dan kalau PATH-nya belum benar perintahnya tidak dikenali. Versi tools juga bisa jadi masalah karena tidak semua cocok satu sama lain. Kadang saya juga hanya mengikuti langkah di tutorial tanpa benar-benar paham fungsi tiap tools.

Untuk mengatasinya, saya akan mengikuti panduan di situs resmi dan mengecek hasil instalasi dengan git --version, node -v, dan java -version. Kalau ada error, saya baca dulu pesan errornya, lalu cari solusinya di dokumentasi atau forum. Saya juga akan mencatat langkah-langkah yang sudah dicoba supaya kalau harus mengulang tidak bingung, dan bertanya ke dosen atau teman kalau masih buntu.

Link Repository: https://github.com/AyuNatasya01/pbp-240511018